

UPCASTING E DOWNCASTING

Java, come altri linguaggi di programmazione, è un linguaggio tipizzato.

Esempio:

```
double f;
```

```
f=4.70; // OK
```

```
f="hello"; // ERRORE
```

```
Car c; //definisco un riferimento ad un oggetto di tipo Car
```

```
c = new Car();
```

c: riferimento di tipo Car che punta ad un oggetto di tipo Car presente nello heap di sistema

new Car(): creazione di un oggetto di tipo Car

Vi sono, però, dei casi particolari...

```
public class Car{...}
```

```
public class SDCar extends Car{...}
```

```
Car c1 = new Car();
```

c1: riferimento di tipo Car che punta ad un oggetto tipo Car

```
SDCar c2 = new SDCar();
```

c2: riferimento di tipo SDCar che punta ad un oggetto tipo SDCar

UPCASTING

```
Car c3 = new SDCar();
```

c3: riferimento di tipo Car che contiene l'indirizzo di un oggetto presente nello heap di sistema di tipo SDCar.

c3 è un riferimento più generico rispetto all'oggetto istanziato.

new: l'operatore che definisce uno spazio nello heap per contenere l'oggetto.

Significato della scrittura precedente: **AVERE UN RIFERIMENTO DI TIPO Car significa che sull'oggetto di tipo SDCar possono essere chiamati solo i metodi definiti nella classe Car.**

Definizione (**UPCASTING**): assegnamento di un riferimento più generico rispetto all'oggetto che è stato creato nella memoria dinamica.

L'upcasting è:

- affidabile
- automatico (non è necessario indicare l'operatore di casting)
- va verso la generalizzazione
- può essere anche esplicito

Esempio di upcasting

```
public class Car{
    boolean isOn;
    String licensePlate;
    void turnOn(){...}
    void turnOff(){...}
}

public class SDCar extends Car{
    boolean selfDriving;
    void turnSDOn(){...}
    void turnSDOff(){...}
}

SDCar c1 = new SDCar(); //riferimento specifico quanto l'oggetto
c1.turnSDOn(); //OK

/* Si può invocare sull'oggetto, tramite il riferimento
c1, il metodo turnSDOn() perché tale metodo è definito
nella classe SDCar.

*/

Car c2 = c1; //UPCASTING

c2: riferimento ad un oggetto di tipo Car che punta ad un oggetto (già presente
nella memoria) puntato dal riferimento c1.

c2.turnSDOn(); //errore a tempo di compilazione
```

c2: riferimento più generico che toglie la possibilità di accedere ai metodi della classe **SDCar**.

Definizione (**DOWNCASTING**): assegnamento di un riferimento più specifico rispetto all'oggetto che è stato creato nella memoria dinamica.

Il downcasting è:

- un'operazione rischiosa e non è effettuata in modo automatico dal compilatore
- Definito in modo esplicito
- Va verso la specializzazione

Esempio di downcasting

```
public class Car{
    boolean isOn;
    String licensePlate;
    void turnOn(){...}
    void turnOff(){...}
}

public class SDCar extends Car{
    boolean selfDriving;
    void turnSDOn(){...}
    void turnSDOff(){...}
}

Car c1 = new SDCar(); //UPCASTING
c1.turnOn(); //OK
SDCar c2 = (SDCar) c1; //DOWNCASTING
```

Il riferimento c1 viene forzato a diventare di tipo SDCar.

L'istruzione `c2.turnSDOn();` non dà errore perché `turnSDOn()` è un metodo specifico definito nella classe `SDCar` e l'oggetto sottostante istanziato è di tipo `SDCar`.

Se c1 non fosse riferito ad un oggetto di tipo SDCar, la scrittura `c2.turnSDOn()` avrebbe prodotto un errore in quanto il metodo `turnSDOn()` non è stato definito sull'oggetto.

ERRORI A TEMPO DI COMPILAZIONE E A TEMPO DI ESECUZIONE DURANTE IL DOWNCASTING

```
public class Car{
    boolean isOn;
    String licensePlate;
    void turnOn(){...}
    void turnOff(){...}
}

public class SDCar extends Car{
    boolean selfDriving;
    void turnSDOn(){...}
    void turnSDOff(){...}
}

Car c1 = new Car(); //OK istanziazione senza casting
c1.turnSDOn(); //ERRORE A TEMPO DI COMPILAZIONE
SDCar c2 = (SDCar) c1; //DOWNCASTING
c2.turnSDOn();
//dalla piattaforma di sviluppo non viene segnalato come errore
```

NOTA BENE:

ERRORE A RUN TIME perché l'oggetto sottostante è sempre di tipo Car e nella classe Car non è stato definito questo metodo.

Per risolvere questo problema, Java mette a disposizione dei programmatori un operatore binario chiamato **instanceof**.

```
Car c3 = new SDCar();  
if(c3 instanceof SDCar){  
    SDCar c4 = (SDCar) c3;  
    c4.turnSDOn();  
}
```

Prima di effettuare il downcasting, si utilizza instanceof per controllare se il riferimento all'oggetto possa essere "castato" al tipo desiderato.

La condizione `if(c3 instanceof SDCar)` verifica se l'oggetto referenziato da c3 è effettivamente un'istanza di SDCar. Se il controllo restituisce **true**, significa che c3 si riferisce ad un oggetto SDCar.

L'operatore **instanceof** in Java è molto utile per verificare se un oggetto è un'istanza di una specifica classe o implementa una certa interfaccia. Questo è particolarmente rilevante nel contesto del **downcasting**, ovvero quando si converte il riferimento ad un oggetto da un tipo più generico (superclasse) a un tipo più specifico (sottoclasse).

Spiegazione esempio precedente:

Car: classe base (rappresenta un'auto generica).

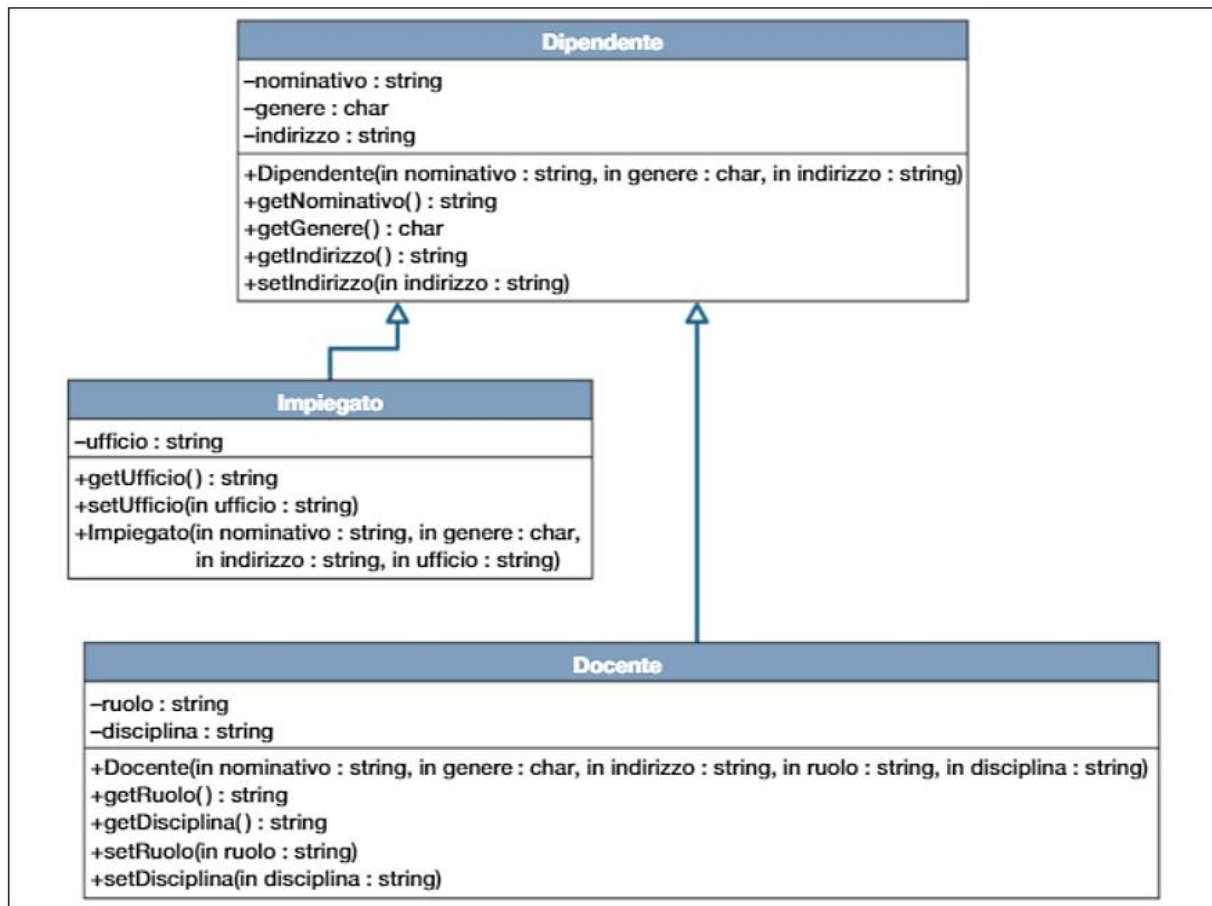
SDCar: sottoclasse di Car (ovvero, un'auto con funzionalità "SD", magari con funzionalità specifiche come il metodo `turnSDOn()`).

turnSDOn(): metodo definito solo in SDCar (e non in Car), che abilita una particolare funzione.

c3: tramite c3, si accede solo ai metodi o proprietà definiti in Car, perché la variabile è tipizzata come Car.

if(c3 instanceof SDCar): il controllo con instanceof serve a verificare il "tipo reale/effettivo" dell'oggetto sottostante alla variabile. La variabile c3 contiene un riferimento di tipo Car, **ma il controllo instanceof conferma che il tipo dell'oggetto è effettivamente SDCar.**

Esempio (diagramma UML a pagina A188 del libro di testo)



UPCASTING IMPLICITO

```
Dipendente dipendente1, dipendente2;
```

```
dipendente1 = new Impiegato("Rossi Mario", 'M', "Via del Mare, 1 - Livorno", "Segreteria");
```

```
dipendente2 = new Docente("Bianchi Maria", 'F', "Via Nazionale, 2 - Parma", "Di ruolo", "Informatica");
```

Nota bene: anche se dipendente1 e dipendente 2 sono due riferimenti ad oggetti di tipo Impiegato e Docente, è possibile accedere ai metodi definiti nella classe Dipendente poiché l'oggetto Docente e l'oggetto Impiegato sono trattati come oggetti della classe base.

UPCASTING ESPLICITO

```
Impiegato impiegato1 = new Impiegato("Rossi Mario", 'M', "Via del Mare, 1 - Livorno", "Segreteria");
```

```
Docente docente1 = new Docente("Bianchi Maria", 'F', "Via Nazionale, 2 - Parma", "Di ruolo",  
"Informatica");
```

```
Dipendente dipendente1 = (Dipendente)impiegato1; (*)
```

```
Dipendente dipendente2 = (Dipendente)docente1;
```

(*) l'oggetto sottostante è di tipo Impiegato e il riferimento all'oggetto è impiegato1.

Con la scrittura (Dipendente) si sta dicendo al compilatore di trattare l'oggetto di tipo Impiegato come un'istanza della classe base Dipendente in quanto il riferimento all'oggetto (impiegato1) viene "forzato" ad essere di tipo Dipendente.

Dopo l'operazione di upcasting potranno essere invocati sull'oggetto solo i metodi definiti nella classe base e non potranno essere chiamati quelli specifici della classe Impiegato.

OSSERVAZIONE:

l'oggetto sottostante (presente nell'heap di sistema) è ancora di tipo Impiegato.

il TIPO EFFETTIVO (runtime type) dell'oggetto non cambia. Anche se l'oggetto viene trattato come istanza della classe Base, in memoria l'oggetto è sempre istanza di Impiegato.

Quindi, in conclusione, i metodi che si possono invocare sull'oggetto di tipo Impiegato sono:

- getNominativo()
- getGenere()
- getIndirizzo()
- setIndirizzo(String indirizzo)

Esempio

```
System.out.println("Nominativo: " + dipendente1.getNominativo()); //OK
```

```
System.out.println("Indirizzo: " + dipendente1.getIndirizzo()); //OK
```

```
dipendente1.getUfficio();
```

```
/*
```

getUfficio() è un metodo non accessibile tramite il riferimento dipendente1 in quanto non definito nella classe Dipendente: errore a compile time

Come si risolve questo errore?

Effettuando il DOWNCASTING ESPLICITO

```
if(dipendente1 instanceof Impiegato){  
    Impiegato impiegato = (Impiegato)dipendente1;  
    System.out.println("Ufficio: " + impiegato.getUfficio());  
}
```

APPROFONDIMENTO: QUANDO USARE L'OPERATORE super**Quando si usa super:**

1. **accesso a metodi della superclasse:** se la sottoclasse ridefinisce un metodo della superclasse e si desidera richiamare il metodo originale, si utilizza super.

Esempio

```
class SuperClass {  
    void display() {  
        System.out.println("Metodo della superclasse");  
    }  
}  
  
class SubClass extends SuperClass {  
    void display() {  
        super.display(); // Richiama il metodo della superclasse  
        System.out.println("Metodo della sottoclasse");  
    }  
}
```

2. **Costruttore della superclasse:** richiamare esplicitamente un costruttore specifico della superclasse.

Esempio:

```
class Libro {  
    String titolo;  
    String autore;  
    // Costruttore della superclasse  
    Book(String titolo, String autore) {  
        this.titolo = titolo;  
        this.autore = autore;  
    }  
}
```

```
void dettagliLibro() {
    System.out.println("Titolo: " + titolo);
    System.out.println("Autore: " + autore);
}

class EBook extends Libro {
    double fileSize; // Dimensione del file in MB
    // Costruttore della sottoclasse
    EBook(String titolo, String autore, double fileSize) {
        super(titolo, autore);
        this.fileSize = fileSize;
    }
    @Override
    void dettagliLibro() {
        //Richiama il metodo della superclasse
        super.dettagliLibro();
        System.out.println("Dimensione del file: " +
            fileSize + " MB");
    }
}

public class Main {
    public static void main(String[] args) {
        EBook ebook = new EBook("Il Grande Gatsby", "F.
            Scott Fitzgerald", 2.5);
        ebook.dettagLibro();
    }
}
```

Quando non si usa super:

1. in presenza di metodi astratti.
2. Nel caso di non sovrascrittura, cioè quando si vuole utilizzare un metodo della classe base nella classe derivata senza effettuare l'override.

Esempio:

```
class Persona{
    public void saluto(){
        System.out.println("Ciao, sono una persona! ");
    }
}

class Studente extends Persona{
    public void salutoStudente(){
        saluto();
        System.out.println("Sono uno studente e uso il
        metodo della classe base. ");
    }
}
```